第2章 一切都是对象

“如果我们说另一种不同的语言，那么我们就会发觉一个有些不同的世界。”

—Luduing Wittgerstein (1889-1951)

尽管Java是基于C++的，但是相比之下，Java是一种更“纯粹”的面向对象程序设计语言。

C++和Java都是混合/杂合型语言。但是，Java的设计者认为这种杂合性并不像在C++中那么重要。杂合型语言允许多种编程风格；C++之所以成为一种杂合型语言主要是因为它支持与C语言的向后兼容。因为C++是C的一个超集，所以势必包括许多C语言不具备的特性，这些特性使C++在某些方面显得过于复杂。

Java语言假设我们只进行面向对象的程序设计。也就是说，在开始用Java进行设计之前，必须将思想转换到面向对象的世界中来。这个入门基本功，可以使你具备使用这样一种编程语言编程的能力，这种语言学习起来更简单，也比许多其他OOP语言更易用。在本章，我们将看到Java程序的基本组成部分，并体会到在Java中（几乎）一切都是对象。

2.1 用引用操纵对象

每种编程语言都有自己的操纵内存中元素的方式。有时候，程序员必须注意将要处理的数据是什么类型。你是直接操纵元素，还是用某种基于特殊语法的间接表示（例如C和C++里的指针）来操纵对象？

所有这一切在Java里都得到了简化。一切都被视为对象，因此可采用单一固定的语法。尽管一切都看作对象，但操纵的标识符实际上是对象的一个“引用”（reference）[⊖]。可以将这一情形想像成用遥控器（引用）来操纵电视机（对象）。只要握住这个遥控器，就能保持与电视机的连接。当有人想改变频道或者减小音量时，实际操控的是遥控器（引用），再由遥控器来调控电视机（对象）。如果想在房间里四处走走，同时仍能调控电视机，那么只需携带遥控器（引用）而不是电视机（对象）。

此外，即使没有电视机，遥控器亦可独立存在。也就是说，你拥有一个引用，并不一定需要有一个对象与它关联。因此，如果想操纵一个词或句子，则可以创建一个**String**引用：

```
String s;
```

但这里所创建的只是引用，并不是对象。如果此时向s发送一个消息，就会返回一个运行时

⊖ 这可能会引起争论。有人认为：“很明显，它是一个指针。”但是这种说法是基于底层实现的某种假设。并且，Java中的引用，在语法上更接近C++的引用而不是指针。本书的第1版中，我选择发明一个新术语“句柄(handle)”来表示这一概念，因为，Java的引用和C++的引用毕竟存在一些重大差异。我当时正在脱离C++阵营，而且也不想使那些已经习惯C++语言的程序员（我想他们将来会是最大的、热衷于Java的群体）感到迷惑。在第2版中，我决定换回这个最为广泛使用的术语——“引用”。并且，那些从C++阵营转换过来的人们，理应更会处理引用，而不是仅仅理解“引用”这个术语，因而他们也会全心全意投入其中的。尽管如此，还是有人不同意用“引用”这个术语。我曾经读到的一本书这样说：“Java所支持的‘按址传递’是完全错误的”，因为Java对象标识符（按那位作者所说）实际上是“对象引用”。并且他接着说任何事物都是“按值传递”的。也许有人会赞成这种精确却让人费解的解释，但我认为我的这种方法可以简化概念上的理解并且不会伤害到任何事物。（好了，那些语言专家可能会说我在撒谎，但我认为我只是提供了一个合适的抽象罢了。）

错误。这是因为此时s实际上没有与任何事物相关联（即，没有电视机）。因此，一种安全的做法是：创建一个引用的同时便进行初始化。

62

```
String s = "asdf";
```

但这里用到了Java语言的一个特性：字符串可以用带引号的文本初始化。通常，必须对对象采用一种更通用的初始化方法。

2.2 必须由你创建所有对象

一旦创建了一个引用，就希望它能与一个新的对象相关联。通常用**new**操作符来实现这一目的。**new**关键字的意思是“给我一个新对象。”所以前面的例子可以写成：

```
String s = new String("asdf");
```

它不仅表示“给我一个新的字符串”，而且通过提供一个初始字符串，给出了怎样产生这个**String**的信息。

当然，除了**String**类型，Java提供了大量过剩的现成类型。重要的是，你可以自行创建类型。事实上，这是Java程序设计中一项基本行为，你会在本书以后章节中慢慢学到。

2.2.1 存储到什么地方

程序运行时，对象是怎么进行放置安排的呢？特别是内存是怎样分配的呢？对这些方面的了解会对你有很大的帮助。有五个不同的地方可以存储数据：

1) **寄存器**。这是最快的存储区，因为它位于不同于其他存储区的地方——处理器内部。但是寄存器的数量极其有限，所以寄存器根据需求进行分配。你不能直接控制，也不能在程序中感觉到寄存器存在的任何迹象（另一方面，C和C++允许您向编译器建议寄存器的分配方式）。

2) **堆栈**。位于通用RAM（随机访问存储器）中，但通过堆栈指针可以从处理器那里获得直接支持。堆栈指针若向下移动，则分配新的内存；若向上移动，则释放那些内存。这是一种快速有效的分配存储方法，仅次于寄存器。创建程序时，Java系统必须知道存储在堆栈内所有项的确切生命周期，以便上下移动堆栈指针。这一约束限制了程序的灵活性，所以虽然某些Java数据存储于堆栈中——特别是对象引用，但是Java对象并不存储于其中。

63

3) **堆**。一种通用的内存池（也位于RAM区），用于存放所有的Java对象。堆不同于堆栈的好处是：编译器不需要知道存储的数据在堆里存活多长时间。因此，在堆里分配存储有很大的灵活性。当需要一个对象时，只需用**new**写一行简单的代码，当执行这行代码时，会自动在堆里进行存储分配。当然，为这种灵活性必须要付出相应的代价：用堆进行存储分配和清理可能比用堆栈进行存储分配需要更多的时间（如果确实可以在Java中像在C++中一样在栈中创建对象）。

4) **常量存储**。常量值通常直接存放在程序代码内部，这样做是安全的，因为它们永远不会被改变。有时，在嵌入式系统中，常量本身会和其他部分隔离开，所以在这种情况下，可以选择将其存放在ROM（只读存储器）中[⊖]。

5) **非RAM存储**。如果数据完全存活于程序之外，那么它可以不受程序的任何控制，在程序没有运行时也可以存在。其中两个基本的例子是流对象和持久化对象。在流对象中，对象转化成字节流，通常被发送给另一台机器。在“持久化对象”中，对象被存放于磁盘上，因此，即使程序终止，它们仍可以保持自己的状态。这种存储方式的技巧在于：把对象转化成可以存放

⊖ 这种存储区的一个例子是字符串池。所有字面常量字符串和具有字符串值的常量表达式都自动是内存限定的，并且会置于特殊的静态存储区中。

在其他媒介上的事物，在需要时，可恢复成常规的、基于RAM的对象。Java提供了对轻量级持久化的支持，而诸如JDBC和Hibernate这样的机制提供了更加复杂的对在数据库中存储和读取对象信息的支持。

64

2.2.2 特例：基本类型

在程序设计中经常用到一系列类型，它们需要特殊对待。可以把它们想像成“基本”类型。之所以特殊对待，是因为**new**将对象存储在“堆”里，故用**new**创建一个对象——特别是小的、简单的变量，往往不是很有效。因此，对于这些类型，Java采取与C和C++相同的方法。也就是说，不用**new**来创建变量，而是创建一个并非是引用的“自动”变量。这个变量直接存储“值”，并置于堆栈中，因此更加高效。

Java要确定每种基本类型所占存储空间的大小。它们的大小并不像其他大多数语言那样随机器硬件架构的变化而变化。这种所占存储空间大小的不变性是Java程序比用其他大多数语言编写的程序更具可移植性的原因之一。

基本类型	大小	最小值	最大值	包装器类型
boolean	—	—	—	**Boolean**
char	16-bit	Unicode 0	Unicode $2^{16}-1$	**Character**
byte	8 bits	-128	+127	**Byte**
short	16 bits	-2^{15}	$+2^{15}-1$	**Short**
int	32 bits	-2^{31}	$+2^{31}-1$	**Integer**
long	64 bits	-2^{63}	$+2^{63}-1$	**Long**
float	32 bits	IEEE754	IEEE754	**Float**
double	64 bits	IEEE754	IEEE754	**Double**
void	—	—	—	**Void**

所有数值类型都有正负号，所以不要去寻找无符号的数值类型。

boolean类型所占存储空间的大小没有明确指定，仅定义为能够取字面值**true**或**false**。

基本类型具有的包装器类，使得可以在堆中创建一个非基本对象，用来表示对应的基本类型。例如：

```
char c = 'x';
Character ch = new Character(c);
```

65

也可以这样用：

```
Character ch = new Character('x');
```

Java SE5的自动包装功能将自动地将基本类型转换为包装器类型：

```
Character ch = 'x';
```

并可以反向转换：

```
char c = ch;
```

包装基本类型的原因将在以后的章节中说明。

高精度数字

Java提供了两个用于高精度计算的类：**BigInteger** 和**BigDecimal**。虽然它们大体上属于“包装器类”的范畴，但二者都没有对应的基本类型。

不过，这两个类包含的方法，提供的操作与对基本类型所能执行的操作相似。也就是说，能作用于**int** 或**float**的操作，也同样能作用于**BigInteger**或 **BigDecimal**。只不过必须以方法调用

方式取代运算符方式来实现。由于这么做复杂了许多，所以运算速度会比较慢。在这里，我们以速度换取了精度。

BigInteger支持任意精度的整数。也就是说，在运算中，可以准确地表示任何大小的整数值，而不会丢失任何信息。

BigDecimal支持任何精度的定点数，例如，可以用它进行精确的货币计算。

关于调用这两个类的构造器和方法的详细信息，请查阅JDK文档。

2.2.3 Java中的数组

几乎所有的程序设计语言都支持数组。在C和C++中使用数组是很危险的，因为C和C++中的数组就是内存块。如果一个程序要访问其自身内存块之外的数组，或在数组初始化前使用内存（程序中常见的错误），都会产生难以预料的后果。

66

Java的主要目标之一是安全性，所以许多在C和C++里困扰程序员的问题在Java里不会再出现。Java确保数组会被初始化，而且不能在它的范围之外被访问。这种范围检查，是以每个数组上少量的内存开销及运行时的下标检查为代价的。但由此换来的是安全性和效率的提高，因此付出的代价是值得的（并且Java有时可以优化这些操作）。

当创建一个数组对象时，实际上就是创建了一个引用数组，并且每个引用都会自动被初始化为一个特定值，该值拥有自己的关键字**null**。一旦Java看到**null**，就知道这个引用还没有指向某个对象。在使用任何引用前，必须为其指定一个对象；如果试图使用一个还是**null**的引用，在运行时将会报错。因此，常犯的数组错误在Java中就可以避免。

还可以创建用来存放基本数据类型的数组。同样，编译器也能确保这种数组的初始化，因为它会将这种数组所占的内存全部置零。

数组将在以后的章节中详细讨论。

2.3 永远不需要销毁对象

在大多数程序设计语言中，变量生命周期的概念，占据了程序设计工作中非常重要的部分。变量需要存活多长时间？如果想要销毁对象，那什么时刻进行呢？变量生命周期的混乱往往会导致大量的程序bug，本节将介绍Java是怎样替我们完成所有的清理工作，从而大大地简化这个问题的。

2.3.1 作用域

大多数过程型语言都有作用域（scope）的概念。作用域决定了在其内定义的变量名的可见性和生命周期。在C、C++和Java中，作用域由花括号的位置决定。例如：

67

```
{
  int x = 12;
  // Only x available
  {
    int q = 96;
    // Both x & q available
  }
  // Only x available
  // q is "out of scope"
}
```

在作用域里定义的变量只可用于作用域结束之前。

任何位于“//”之后到行末的文字都是注释。

缩排格式使Java代码更易于阅读。由于Java是一种自由格式（free-form）的语言，所以，空格、制表符、换行都不会影响程序的执行结果。

尽管以下代码在C和C++中是合法的，但是在Java中却不能这样书写：

```
{
  int x = 12;
  {
    int x = 96; // Illegal
  }
}
```

编译器将会报告变量x已经定义过。所以，在C和C++里将一个较大作用域的变量“隐藏”起来的做法，在Java里是不允许的。因为Java设计者认为这样做会导致程序混乱。

2.3.2 对象的作用域

Java对象不具备和基本类型一样的生命周期。当用**new**创建一个Java对象时，它可以存活于作用域之外。所以假如你采用代码

```
{
  String s = new String("a string");
} // End of scope
```

引用s在作用域终点就消失了。然而，s指向的**String**对象仍继续占据内存空间。在这一小段代码中，我们无法在这个作用域之后访问这个对象，因为对它唯一的引用已超出了作用域的范围。在后继章节中，读者将会看到：在程序执行过程中，怎样传递和复制对象引用。 68

事实证明，由**new**创建的对象，只要你需要，就会一直保留下去。这样，许多C++编程问题在Java中就完全消失了。在C++中，你不仅必须要确保对象的保留时间与你需要这些对象的时间一样长，而且还必须在你使用完它们之后，将其销毁。

这样便带来一个有趣的问题。如果Java让对象继续存在，那么靠什么才能防止这些对象填满内存空间，进而阻塞你的程序呢？这正是C++里可能会发生的问题。这也是Java神奇之所在。Java有一个*垃圾回收器*，用来监视用**new**创建的所有对象，并辨别那些不会再被引用的对象。随后，释放这些对象的内存空间，以便供其他新的对象使用。也就是说，你根本不必担心内存回收的问题。你只需要创建对象，一旦不再需要，它们就会自行消失。这样做就消除了这类编程问题（即“内存泄漏”），这是由于程序员忘记释放内存而产生的问题。

2.4 创建新的数据类型：类

如果一切都是对象，那么是什么决定了某一类对象的外观与行为呢？换句话说，是什么确定了对象的类型？你可能期望有一个名为“type”的关键字，当然它必须还要有相应的含义。然而，从历史发展角度来看，大多数面向对象的程序设计语言习惯用关键字**class**来表示“我准备告诉你一种新类型的对象看起来像什么样子”。**class**这个关键字（以后会频繁使用，本书以后就不再用粗体字表示）之后紧跟着的是新类型的名称。例如：

```
class ATypeName { /* Class body goes here */ }
```

这就引入了一种新的类型，尽管类主体仅包含一条注释语句（星号和斜杠以及其中的内容就是注释，本章后面再讨论）。因此，你还不能用它做太多的事情。然而，你已经可以用**new**来创建这种类型的对象：

```
ATypeName a = new ATypeName();
```

69

但是，在定义它的所有方法之前，还没有办法能让它去做更多的事情（也就是说，不能向它发送任何有意义的消息）。

2.4.1 字段和方法

一旦定义了一个类（在Java中你所做的全部工作就是定义类，产生那些类的对象，以及发送消息给这些对象），就可以在类中设置两种类型的元素：*字段*（有时被称作*数据成员*）和*方法*（有时被称作*成员函数*）。字段可以是任何类型的对象，可以通过其引用与其进行通信；也可以是基本类型中的一种。如果字段是对某个对象的引用，那么必须初始化该引用，以便使其与一个实际的对象（如前所述，使用**new**来实现）相关联。

每个对象都有用来存储其字段的空间；普通字段不能在对象间共享。下面是一个具有某些字段的类：

```
class DataOnly {
  int i;
  double d;
  boolean b;
}
```

尽管这个类除了存储数据之外什么也不能做，但是仍旧可以像下面这样创建它的一个对象：

```
DataOnly data = new DataOnly();
```

可以给字段赋值，但首先必须知道如何引用一个对象的成员。具体的实现为：在对象引用的名称之后紧接着一个句点，然后再接着是对象内部的成员名称：

```
objectReference.member
```

例如：

```
data.i = 47;
data.d = 1.1;
data.b = false;
```

想修改的数据也有可能位于对象所包含的其他对象中。在这种情况下，只需要再使用连接句点即可。例如：

70

```
myPlane.leftTank.capacity = 100;
```

DataOnly类除了保存数据外没别的用处，因为它没有任何成员方法。如果想了解成员方法的运行机制，就得先了解*参数*和*返回值*的概念，稍后将对此作简略描述。

基本成员默认值

若类的某个成员是基本数据类型，即使没有进行初始化，Java也会确保它获得一个默认值，如右表所示：

基本类型	默认值
boolean	**false**
char	**'\u0000' (null)**
byte	**(byte)0**
short	**(short)0**
int	**0**
long	**0L**
float	**0.0f**
double	**0.0d**

当变量作为类的成员使用时，Java才确保给定其默认值，以确保那些是基本类型的成员变量得到初始化（C++没有此功能），防止产生程序错误。但是，这些初始值对你的程序来说，可能是不正确的，甚至是不合法的。所以最好明确地对变量进行初始化。

然而上述确保初始化的方法并不适用于“局部”变量（即并非某个类的字段）。因此，如果在某个方法定义中有

```
int x;
```

那么变量**x**得到的可能是任意值（与C 和C++中一样），而不会被自动初始化为零。所以在使用**x**前，应先对其赋一个适当的值。如果忘记了这么做，Java会在编译时返回一个错误，告诉你此变量没有初始化，这正是Java优于C++的地方。（许多C++编译器会对未初始化变量给予警告，而Java则视为是错误）。

71

2.5 方法、参数和返回值

许多程序设计语言（像C和C++）用函数这个术语来描述命名子程序；而在Java里却常用方法这个术语来表示“做某些事情的方式”。实际上，继续把它看作是函数也无妨。尽管这只是用词上的差别，但本书将沿用Java的惯用法，即用术语“方法”而不是“函数”来描述。

Java的方法决定了一个对象能够接收什么样的消息。方法的基本组成部分包括：名称、参数、返回值和方法体。下面是它最基本的形式：

```
ReturnType methodName( /* Argument list */ ) {
  /* Method body */
}
```

返回类型描述的是在调用方法之后从方法返回的值。参数列表给出了要传给方法的信息的类型和名称。方法名和参数列表（它们合起来被称为“方法签名”）唯一地标识出某个方法。

Java中的方法只能作为类的一部分来创建。方法只有通过对象才能被调用[⊖]，且这个对象必须能执行这个方法调用。如果试图在某个对象上调用它并不具备的方法，那么在编译时就会得到一条错误消息。通过对象调用方法时，需要先列出对象名，紧接着是句点，然后是方法名和参数列表。如：

```
objectName.methodName(arg1, arg2, arg3);
```

例如，假设有一个方法**f()**，不带任何参数，返回类型是**int**。如果有个名为**a**的对象，可以通过它调用**f()**，那么就可以这样写：

```
int x = a.f();
```

返回值的类型必须要与**x**的类型兼容。 72

这种调用方法的行为通常被称为发送消息给对象。在上面的例子中，消息是**f()**，对象是**a**。面向对象的程序设计通常简单地归纳为“向对象发送消息”。

2.5.1 参数列表

方法的参数列表指定要传递给方法什么样的信息。正如你可能料想的那样，这些信息像Java中的其他信息一样，采用的都是对象形式。因此，在参数列表中必须指定每个所传递对象的类型及名字。像Java中任何传递对象的场合一样，这里传递的实际上也是引用[⊜]，并且引用的类型必须正确。如果参数被设为**String**类型，则必须传递一个**String**对象；否则，编译器将抛出错误。

假设某个方法接受**String**为其参数，下面是其具体定义，它必须置于某个类的定义内才能被正确编译。

```
int storage(String s) {
  return s.length() * 2;
}
```

此方法告诉你，需要多少个字节才能容纳一个特定的**String**对象中的信息（字符串中的每个字符的尺寸都是16位或2个字节，以此来提供对Unicode字符集的支持）。此方法的参数类型是**String**，参数名是**s**。一旦将**s**传递给此方法，就可以把他当作其他对象一样进行处理（可以给它传递消息）。在这里，**s**的**length()**方法被调用，它是**String**类提供的方法之一，会返回字符串包

⊖ 稍后将会学到**static**方法，它是针对类调用的，并不依赖于对象的存在。

⊜ 对于前面所提到的特殊数据类型 **boolean**、**char**、**byte**、**short**、**int**、**long**、**float**和**double**来说是一个例外。通常，尽管传递的是对象，而实际上传递的是对象的引用。

含的字符数。

通过上面的例子，还可以了解到**return**关键字的用法，它包括两方面：首先，它代表“已经做完，离开此方法”。其次，如果此方法产生了一个值，这个值要放在**return**语句后面。在这个例子中，返回值是通过计算**s.length() * 2**这个表达式得到的。

73

你可以定义方法返回任意想要的类型，如果不想返回任何值，可以指示此方法返回**void**(空)。下面是一些例子：

```
boolean flag() { return true; }
double naturalLogBase() { return 2.718; }
void nothing() { return; }
void nothing2() {}
```

若返回类型是**void**，**return**关键字的作用只是用来退出方法。因此，没有必要到方法结束时才离开，可在任何地方返回。但如果返回类型不是**void**，那么无论在何处返回，编译器都会强制返回一个正确类型的返回值。

到此为止，读者或许觉得：程序似乎只是一系列带有方法的对象组合，这些方法以其他对象为参数，并发送消息给其他对象。大体上确实是这样，但在以后章节中，读者将会学到怎样在一个方法内进行判断，做一些更细致的底层工作。至于本章，读者只需要理解消息发送就足够了。

2.6 构建一个Java程序

在构建自己的第一个Java程序前，还必须了解其他一些问题。

2.6.1 名字可见性

名字管理对任何程序设计语言来说，都是一个重要问题。如果在程序的某个模块里使用了一个名字，而其他人在这个程序的另一个模块里也使用了相同的名字，那么怎样才能区分这两个名字并防止二者互相冲突呢？这个问题在C语言中尤其严重，因为程序往往包含许多难以管理的名字。C++类（Java类基于此）将函数包于其内，从而避免了与其他类中的函数名相冲突。然而，C++仍允许全局数据和全局函数的存在，所以还是有可能发生冲突。为了解决这个问题，C++通过几个关键字引入了*名字空间*的概念。

Java采用了一种全新的方法来避免上述所有问题。为了给一个类库生成不会与其他名字混淆的名字，Java设计者希望程序员反过来使用自己的Internet域名，因为这样可以保证它们肯定是独一无二的。由于我的域名是**MindView.net**，所以我的各种奇奇怪怪的应用工具类库就被命名为**net.mindview.utility.foibles**。反转域名后，句点就用来代表子目录的划分。

74

在Java 1.0和Java 1.1中，扩展名**com**、**edu**、**org**、**net**等约定为大写形式。所以上面的库名应该写成**NET.mindview.utility.foibles**。然而，在Java 2开发到一半时，设计者们发现这样做会引起一些问题，因此，现在整个包名都是小写了。

这种机制意味着所有的文件都能够自动存活于它们自己的名字空间内，而且同一个文件内的每个类都有唯一的标识符——Java语言本身已经解决了这个问题。

2.6.2 运用其他构件

如果想在自己的程序里使用预先定义好的类，那么编译器就必须知道怎么定位它们。当然，这个类可能就在发出调用的那个源文件中，在这种情况下，就可以直接使用这个类——即使这个类在文件的后面才会被定义（Java消除了所谓的“向前引用”问题）。

如果那个类位于其他文件中，又会怎样呢？你可能会认为编译器应该有足够的智慧，能够

直接找到它的位置，但事实并非如此。想像下面的情况，如果你想使用某个特定名字的类，但其定义却不止一份（假设这些定义各不相同）。更糟糕的是，假设你正在写一个程序，在构建过程中，你想将某个新类添加到类库中，但却与已有的某个类名冲突。

为了解决这个问题，必须消除所有可能的混淆情况。为实现这个目的，可以使用关键字**import**来准确地告诉编译器你想要的类是什么。**import**指示编译器导入一个包，也就是一个类库（在其他语言中，一个库不仅包含类，还可能包括方法和数据；但是Java中所有的代码都必须写在类里）。

大多时候，我们使用与编译器附在一起的Java标准类库里的构件。有了这些构件，你就不必写一长串的反转域名。举例来说，只须像下面这么书写就行了：

```
import java.util.ArrayList;
```

75

这行代码告诉编译器，你想使用Java的**ArrayList**类。但是，**util**包含了数量众多的类，有时你想使用其中的几个，同时又不想明确地逐一声明；那么你很容易使用通配符“*”来达到这个目的：

```
import java.util.*;
```

这种一次导入一群类的方式比一个一个地导入类的方式更常用。

2.6.3 static 关键字

通常来说，当创建类时，就是在描述那个类的对象的外观与行为。除非用**new**创建那个类的对象，否则，实际上并未获得任何对象。执行**new**来创建对象时，数据存储空间才被分配，其方法才供外界调用。

有两种情形用上述方法是无法解决的。一种情形是，只想为某特定域分配单一存储空间，而不去考虑究竟要创建多少对象，甚至根本就不创建任何对象。另一种情形是，希望某个方法不与包含它的类的任何对象关联在一起。也就是说，即使没有创建对象，也能够调用这个方法。

通过**static**关键字可以满足这两方面的需要。当声明一个事物是**static**时，就意味着这个域或方法不会与包含它的那个类的任何对象实例关联在一起。所以，即使从未创建某个类的任何对象，也可以调用其**static**方法或访问其**static**域。通常，你必须创建一个对象，并用它来访问数据或方法。因为非**static**域和方法必须知道它们一起运作的特定对象[⊖]。

有些面向对象语言采用*类数据*和*类方法*两个术语，代表那些数据和方法只是作为整个类，而不是类的某个特定对象而存在的。有时，一些Java文献里也用到这两个术语。

76

只须将**static**关键字放在定义之前，就可以将字段或方法设定为**static**。例如，下面的代码就生成了一个**static**字段，并对其进行了初始化：

```
class StaticTest {
  static int i = 47;
}
```

现在，即使你创建了两个**StaticTest**对象，**StaticTest.i**也只有一份存储空间，这两个对象共享同一个**i**。再看看下面代码：

```
StaticTest st1 = new StaticTest();
StaticTest st2 = new StaticTest();
```

在这里，**st1.i** 和 **st2.i**指向同一存储空间，因此它们具有相同的值48。

引用**static**变量有两种方法。如前例所示，可以通过一个对象去定位它，如**st2.i**；也可以通

⊖ 当然，由于在用static方法前不需要创建任何对象；所以对于static方法，不能简单地通过调用其他非static域或方法而没有指定某个命名对象，来直接访问非static域或方法（因为非static域或方法必须与某一特定对象关联）。

过其类名直接引用，而这对于非静态成员则不行。

```
StaticTest.i++;
```

其中，++运算符对变量进行递加操作。此时，**st1.i** 和 **st2.i**仍具有相同的值48。

使用类名是引用static变量的首选方式，这不仅是因为它强调了变量的static结构，而且在某些情况下它还为编译器进行优化提供了更好的机会。

类似逻辑也应用于静态方法。既可以像其他方法一样，通过一个对象来引用某个静态方法，也可以通过特殊的语法形式**ClassName.method()**加以引用。定义静态方法的方式也与定义静态变量的方式相似：

```
class Incrementable {
  static void increment() { StaticTest.i++; }
}
```

可以看到，**Incrementable** 的**increment()**方法通过++运算符将静态数据**i**递加。可以采用典型的方式，通过对象来调用**increment()**：

77

```
Incrementable sf = new Incrementable();
sf.increment();
```

或者，因为**increment()**是一个静态方法，所以也可以通过它的类直接调用：

```
Incrementable.increment();
```

尽管当**static**作用于某个字段时，肯定会改变数据创建的方式（因为一个**static**字段对每个类来说都只有一份存储空间，而非**static**字段则是对每个对象有一个存储空间），但是如果**static**作用于某个方法，差别却没有那么大。**static**方法的一个重要用法就是在不创建任何对象的前提下就可以调用它。正如我们将会看到的那样，这一点对定义**main()**方法很重要，这个方法是运行一个应用时的入口点。

和其他任何方法一样，**static**方法可以创建或使用与其类型相同的被命名对象，因此，**static**方法常常拿来做“牧羊人”的角色，负责看护与其隶属同一类型的实例群。

2.7 你的第一个Java程序

最后，让我们编写第一个完整的程序。此程序开始是打印一个字符串，然后是打印当前日期，这里用到了Java标准库里的**Date**类。

```
// HelloDate.java
import java.util.*;

public class HelloDate {
  public static void main(String[] args) {
    System.out.println("Hello, it's: ");
    System.out.println(new Date());
  }
}
```

在每个程序文件的开头，必须声明**import**语句，以便引入在文件代码中需要用到的额外类。注意，在这里说它们“额外”，是因为有一个特定类会自动被导入到每一个Java文件中：**java.lang**。打开你的Web浏览器，查找Sun公司提供的文档（若没有从http://java.sun.com[⊖]下载JDK文档，现在开始下载。注意，这个文档并没有随JDK一起打包，你必须专门去下载它）。在包列表里，可以看到Java配套提供的各种类库。请点击其中的**java.lang**，就会显示出这个类库所

78

⊖ Sun提供的Java编译器和文档总是在定期地变化，所以获取它们的最佳方式就是直接从Sun处获得。通过自己下载这些资料，你可以获得最新的版本。

包含的全部类的列表。由于**java.lang**是默认导入到每个Java文件中的，所以它的所有类都可以被直接使用。**java.lang**里没有**Date**类，所以必须导入另外一个类库才能使用它。若不知某个特定类在哪个类库里，可在Java文档中选择“Tree”，便可以看到Java配套提供的每一个类。接下来，用浏览器的“查找”功能查找**Date**。这样就可以发现它以**java.util.Date**的形式被列了出来。于是我们知道它位于**util**类库中，并且必须书写**import java.util.*** 才能使用**Date**类。

现在返回文档最开头的部分，选择**java.lang**，接着是**system**，可以看到**system**类有许多属性；若选择**out**，就会发现它是一个静态 **PrintStream**对象。因为是静态的，所以不需要创建任何东西，**out**对象便已经存在了，只须直接使用即可。但我们能够用**out**对象做些什么事情，是由它的类型**PrintStream**决定的。**PrintStream**在描述文档中是以超链接形式显示，所以很方便进行查看，只须点击它，就可以看到能够为**PrintStream**调用的所有方法。方法的数量不少，本书后面再详加讨论。现在我们只对**println()**方法感兴趣，它的实际作用是“将我给你的数据打印到控制台，完成后换行”。因此，在任何Java程序中，一旦需要将某些数据打印到控制台，就可以这样写：

```
System.out.println("A String of things");
```

类的名字必须和文件名相同。如果你像现在这样创建一个独立运行的程序，那么文件中必须存在某个类与该文件同名（否则，编译器会报错），且那个类必须包含一个名为**main()**的方法，形式如下所示：

```
public static void main(String[] args) {
```

其中，**public**关键字意指这是一个可由外部调用的方法（第5章将详细描述）。**main()**方法的参数是一个**String**对象的数组。在这个程序中并未用到**args**，但是Java编译器要求必须这样做，因为**args**要用来存储命令行参数。

打印日期的这行代码很是有趣的： 79

```
System.out.println(new Date());
```

在这里，传递的参数是一个**Date**对象，一旦创建它之后，就可以直接将它的值（它被自动转换为**String**类型）发送给**println()**。当这条语句执行完毕后，**Date**对象就不再被使用，而垃圾回收器会发现这一情况，并在任意时候将其回收。因此，我们就没必要去关心怎样清理它了。

当你阅读从http://java.sun.com下载的JDK文档时，将会发现**System**有许多其他的方法，使得你可以去创造很多有趣的效果（Java最强大的优势之一就是它具有庞大的标准类库集）。例如：

```
//: object/ShowProperties.java

public class ShowProperties {
  public static void main(String[] args) {
    System.getProperties().list(System.out);
    System.out.println(System.getProperty("user.name"));
    System.out.println(
      System.getProperty("java.library.path"));
  }
} ///:~
```

main()的第一行将显示从运行程序的系统中获取的所有“属性”，因此它可以向你提供环境信息。**list()**方法将结果发送给它的参数：**System.out**。在本书后面的章节中你将会看到，你可以把结果发送到任何地方，例如发送到文件中。你还可以询问具体的属性，例如在本例中，我们查询了用户名和**java.library.path**（在程序开头和结尾处不同寻常的注释将在稍后进行解释。）

2.7.1 编译和运行

要编译、运行这个程序以及本书中其他所有的程序，首先必须要有一个Java开发环境。目前，有相当多的第三方厂商提供开发环境，但是在本书中，我假设使用的是Sun免费提供的JDK 80

(Java Developer's Kit, Java开发人员工具包）开发环境。若使用其他的开发系统[⊖]，请查找该系统的相应文档，以便决定怎样编译和运行程序。

请登录到http://java.sun.com网站，那里会有相关的信息和链接，引导读者下载和安装与自己的机器平台相兼容的JDK。

安装好JDK后，还需要设定好路径信息，以确保计算机能找到**javac**和 **java**这两个文件。然后请下载并解压本书提供的源代码（从www.MindView.net处可以获得），它会为书中每一章自动创建一个子目录。请转到**object**子目录下，并键入：

```
javac HelloDate.java
```

正常情况下，这行命令不会产生任何响应。如果有任何错误消息返回给你，就说明还没能正确安装好JDK，需进一步检查并找出问题所在。

如果没有返回任何回应消息，在命令提示符下键入：

```
java HelloDate
```

接着，便可看到程序中的消息和当天日期被输出。

这个过程也是本书中每一个程序的编译和运行过程。然而，读者还会看到在本书所附源代码中，每一章都有一名为**build.xml**的文件，该文件提供一个“ant”命令，用于自动构建该章的所有文件。Build文件和Ant（以及在哪里下载）在http://MindView.net/Books/BetterJava所提供的补充材料中进行了完备而详细的讨论。一旦安装好Ant（可从http://jakarta.apache.org/ant下载），便可直接在命令行提示符下键入**ant**来编译和运行每一章的程序了。如果尚未安装Ant，只要手工键入javac和java命令即可安装。

2.8 注释和嵌入式文档

Java里有两种注释风格。一种是传统的C语言风格的注释——C++也继承了这种风格。此种注释以“/*”开始，随后是注释内容，并可跨越多行，最后以“*/”结束。注意，许多程序员在连续的注释内容的每一行都以一个“*”开头，所以经常看到像下面的写法：

81

```
/* This is a comment
 * that continues
 * across lines
 */
```

但请记住，进行编译时，/*和*/之间的所有东西都会被忽略，所以上述注释与下面这段注释并没有什么两样：

```
/* This is a comment that
continues across lines */
```

第二种风格的注释也源于C++。这种注释是“单行注释”，以一个“//”起头，直到句末。这种风格的注释因为书写容易，所以更方便、更常用。你无需在键盘上寻找“/”，再寻找“*”（而只需按两次同样的键），而且不必考虑结束注释。下面是这类注释的例子：

```
// This is a one-line comment
```

2.8.1 注释文档

代码文档撰写的最大问题，大概就是对文档的维护了。如果文档与代码是分离的，那么在

⊖ IBM的jikes编译器也是一种常用的编译器，它比Sun的 javac快得多（尽管你可以用Ant来构建一组文件，但是这不会有太大的差异）。另外还有很多创建Java编译器、运行时环境和类库的开源项目。

每次修改代码时，都需要修改相应的文档，这会成为一件相当乏味的事情。解决的方法似乎很简单：将代码同文档“链接”起来。为达到这个目的，最简单的方法是将所有东西都放在同一个文件内。然而，为实现这一目的，还必须使用一种特殊的注释语法来标记文档；此外还需一个工具，用于提取那些注释，并将其转换成有用的形式。这正是Java所做的。

javadoc便是用于提取注释的工具，它是JDK安装的一部分。它采用了Java编译器的某些技术，查找程序内的特殊注释标签。它不仅解析由这些标签标记的信息，也将毗邻注释的类名或方法名抽取出来。如此，我们就可以用最少的工作量，生成相当好的程序文档。

javadoc输出的是一个HTML文件，可以用Web浏览器查看。这样，该工具就使得我们只需创建和维护单一的源文件，并能自动生成有用的文档。有了javadoc，就有了创建文档的简明直观的标准；我们可以期望、甚至要求所有的Java类库都提供相关的文档。 82

此外，如果想对javadoc处理过的信息执行特殊的操作（例如，产生不同格式的输出），那么可以通过编写你自己的被称为“doclets”的javadoc处理器来实现。关于doclets在http//MindView.net/Books/Better Java所提供的补充材料中进行了介绍。

下面仅对基本的javadoc进行简单介绍和概述。全面翔实的描述可从java.sun.com提供的、可下载的JDK文档中找到。（注意此文档并没有与JDK一块打包，需单独下载。）解压缩该文档之后，查阅“tooldocs”子目录（或点击“tooldocs”链接）。

2.8.2 语法

所有javadoc命令都只能在“/**”注释中出现，和通常一样，注释结束于“*/”。使用javadoc的方式主要有两种：嵌入HTML，或使用“文档标签”。*独立文档标签*是一些以“@”字符开头的命令，且要置于注释行的最前面（但是不算前导“*”之后的最前面）。而“*行内文档标签*”则可以出现在javadoc注释中的任何地方，它们也是以“@”字符开头，但要括在花括号内。

共有三种类型的注释文档，分别对应于注释位置后面的三种元素：类、域和方法。也就是说，类注释正好位于类定义之前；域注释正好位于域定义之前；而方法注释也正好位于方法定义的前面。如下面这个简单的例子所示：

```
//: object/Documentation1.java
/** A class comment */
public class Documentation1 {
  /** A field comment */
  public int i;
  /** A method comment */
  public void f() {}
} ///:~
```

注意，javadoc只能为**public**（公共）和**protected**（受保护）成员进行文档注释。**private**（私有）和包内可访问成员（参阅第5章）的注释会被忽略掉，所以输出结果中看不到它们（不过可以用-**private**进行标记，以便把**private**成员的注释也包括在内）。这样做是有道理的，因为只有**public**和**protected**成员才能在文件之外被使用，这是客户端程序员所期望的。 83

上述代码的输出结果是一个HTML文件，它与其他Java文档具有相同的标准格式。因此，用户会非常熟悉这种格式，从而方便地导航到用户自己设计的类。输入上述代码，然后通过javadoc处理产生HTML文件，最后通过浏览器观看生成的结果，这样做是非常值得的。

2.8.3 嵌入式HTML

javadoc通过生成的HTML文档传送HTML命令，这使你能够充分利用HTML。当然，其主要目的还是为了对代码进行格式化，例如：

```
//: object/Documentation2.java
```

```
/**
* <pre>
* System.out.println(new Date());
* </pre>
*/
///:~
```

也可以像在其他Web文档中那样运用HTML，对普通文本按照你自己所描述的进行格式化：

```
//: object/Documentation3.java
/**
* You can <em>even</em> insert a list:
* <ol>
* <li> Item one
* <li> Item two
* <li> Item three
* </ol>
*/
///:~
```

注意，在文档注释中，位于每一行开头的星号和前导空格都会被javadoc丢弃。javadoc会对所有内容重新格式化，使其与标准的文档外观一致。不要在嵌入式HTML中使用标题标签，例如**<h1>**或**<hr>**，因为javadoc会插入自己的标题，而你的标题可能同它们发生冲突。

84

所有类型的注释文档——类、域和方法——都支持嵌入式HTML。

2.8.4 一些标签示例

这里将介绍一些可用于代码文档的javadoc标签。在使用javadoc处理重要事情之前，应该先到JDK文档那里查阅javadoc参考，以学习javadoc的各种不同的使用方法。

1. @see：引用其他类

@see标签允许用户引用其他类的文档。javadoc会在其生成的HTML文件中，通过**@see**标签链接到其他文档。格式如下：

```
@see classname
@see fully-qualified-classname
@see fully-qualified-classname#method-name
```

上述每种格式都会在生成的文档中加入一个具有超链接的“See Also”（参见）条目。但是javadoc不会检查你所提供的超链接是否有效。

2. {@link package.class#member label}

该标签与@see极其相似，只是它用于行内，并且是用“label”作为超链接文本而不用“See Also”。

3. {@docRoot}

该标签产生到文档根目录的相对路径，用于文档树页面的显式超链接。

4. {@inheritDoc}

该标签从当前这个类的最直接的基类中继承相关文档到当前的文档注释中。

5. @version

该标签的格式如下：

85

```
@version version-information
```

其中，“**version-information**”可以是任何你认为适合包含在版本说明中的重要信息。如果javadoc命令行使用了“**-version**”标记，那么就从生成的HTML文档中特别提取出版本信息。

6. @author

该标签的格式如下：

```
@author author-information
```

其中，**author-information**一看便知是你的姓名，但是也可以包括电子邮件地址或者其他任何适宜的信息。如果javadoc命令行使用了**-author**标记，那么就从生成的HTML文档中特别提取作者信息。

可以使用多个标签，以便列出所有作者，但是它们必须连续放置。全部作者信息会合并到同一段落，置于生成的HTML中。

7. @since

该标签允许你指定程序代码最早使用的版本，可以在HTML Java文档中看到它被用来指定所用的JDK版本的情况。

8. @param

该标签用于方法文档中，形式如下：

```
@param parameter-name description
```

其中，**parameter-name**是方法的参数列表中的标识符，**description**是可延续数行的文本，终止于新的文档标签出现之前。可以使用任意多个这种标签，大约每个参数都有一个这样的标签。

9. @return

该标签用于方法文档，格式如下：

```
@return description
```

其中，“**description**”用来描述返回值的含义，可以延续数行。

86

10. @throws

“异常”将在第9章论述。简言之，它们是由于某个方法调用失败而“抛出”的对象。尽管在调用一个方法时，只出现一个异常对象，但是某个特殊方法可能会产生任意多个不同类型的异常，所有这些异常都需要进行说明。所以，异常标签的格式如下：

```
@throws fully-qualified-class-name description
```

其中**fully-qualified-class-name**给出一个异常类的无歧义的名字，而该异常类在别处定义。**description**（同样可以延续数行）告诉你为什么此特殊类型的异常会在方法调用中出现。

11. @deprecated

该标签用于指出一些旧特性已由改进的新特性所取代，建议用户不要再使用这些旧特性，因为在不久的将来它们很可能会被删除。如果使用一个标记为**@deprecated**的方法，则会引起编译器发布警告。

在Java SE5中，Javadoc标签@deprecated已经被@Deprecated注解所替代（我们将在第20章中学习相关的知识。）

2.8.5 文档示例

下面再回到第一个Java程序，但是这次加上了文档注释：

```
//: object/HelloDate.java
import java.util.*;

/** The first Thinking in Java example program.
 * Displays a string and today's date.
 * @author Bruce Eckel
 * @author www.MindView.net
 * @version 4.0
*/
public class HelloDate {
  /** Entry point to class & application.
```

87

```
    * @param args array of string arguments
    * @throws exceptions No exceptions thrown
    */
    public static void main(String[] args) {
      System.out.println("Hello, it's: ");
      System.out.println(new Date());
    }
  } /* Output: (55% match)
  Hello, it's:
  Wed Oct 05 14:39:36 MDT 2005
  *///:~
```

第一行采用我自己独特的方法，用一个“:”作为特殊记号说明这是包含源文件名的注释行。该行包含文件的路径信息（此时，**object**代表本章），随后是文件名。最后一行也是一行注释，这个“///:~”标志源代码清单的结束。自此，在通过编译器和执行检查后，文档就可以自动更新成本书的文本。

/***Output**标签表示输出的开始部分将由这个文件生成，通过这种形式，它会被自动地测试以验证其准确性。在本例中，（55% match）在向测试系统说明程序的每一次运行和下一次运行的输出存在着很大的差异，因此它们与这里列出的输出预期只有55%的相关性。本书中能够产生输出的大部分示例都包含这种注释方式的输出，因此你可以查看它们的运行输出，并知晓其正确性。

2.9 编码风格

在“Java编程语言编码约定”[⊖]中，代码风格是这样规定的：类名的首字母要大写；如果类名由几个单词构成，那么把它们并在一起（也就是说，不要用下划线来分隔名字），其中每个内部单词的首字母都采用大写形式。例如：

```
class AllTheColorsOfTheRainbow { // ...
```

88

这种风格有时称作“驼峰风格”。几乎其他所有内容——方法、字段（成员变量）以及对象引用名称等，公认的风格与类的风格一样，只是标识符的第一个字母采用小写。例如：

```
class AllTheColorsOfTheRainbow {
  int anIntegerRepresentingColors;
  void changeTheHueOfTheColor(int newHue) {
    // ...
  }
  // ...
}
```

当然，用户还必须键入所有这些长名字，并且不能输错，因此，一定要格外仔细。

Sun程序库中的Java代码也采用本书摆放开、闭花括号的方式。

2.10 总结

通过本章的学习，大家已接触相当多的关于如何编写一个简单程序的Java编程知识。此外，对Java语言及它的一些基本思想也有了一个总体认识。然而到目前为止，所有示例都是“这样做，再那样做，接着再做另一些事情”这种形式。如果想让程序做出选择，例如，“假如所做的结果是红色，就那样做；否则，就做另一些事情”。这将又怎样进行呢？Java对此种基本编程行为所提供的支持，将会在下一章讲述。

⊖ 网址是：http://java.sun.com/docs/codeconv/index.html。为了节省本书和课堂演示的篇幅，没有遵循约定中的全部条款，但是你将会看到我在这里所使用的风格尽可能地与Java标准相匹配。

2.11 练习

所选习题的答案都可以在名为“The Thinking in Java Annotated Solution Guide”的电子文档中找到，读者可以从www.MindView.net处购买此文档。

练习1：(2) 创建一个类，它包含一个**int**域和一个**char**域，它们都没有被初始化，将它们的值打印出来，以验证Java执行了默认初始化。

练习2：(1) 参照本章的**HelloDate.java**这个例子，创建一个“Hello，World”程序，该程序只要输出这句话即可。你所编写的类里只需一个方法（即“main”方法，在程序启动时被执行）。记住要把它设为**static**形式，并指定参数列表—即使根本不会用到这个列表。用**javac**进行编译，再用**java**运行它。如果你使用的是不同于JDK的开发环境，请了解如何在你的环境中进行编译和运行。 89

练习3：(1) 找出含有**ATypeName**的代码段，将其改写成完整的程序，然后编译、运行。

练习4：(1) 将**DataOnly**代码段改写成一个程序，然后编译、运行。

练习5：(1) 修改前一个练习，将**DataOnly**中的数据在**main()**方法中赋值并打印出来。

练习6：(2) 编写一个程序，让它含有本章所定义的**storage()**方法的代码段，并调用之。

练习7：(1) 将**Incrementable**的代码段改写成一个完整的可运行程序。

练习8：(3) 编写一个程序，展示无论你创建了某个特定类的多少个对象，这个类中的某个特定的static域只有一个实例。

练习9：(2) 编写一个程序，展示自动包装功能对所有的基本类型和包装器类型都起作用。

练习10：(2) 编写一个程序，打印出从命令行获得的三个参数。为此，需要确定命令行数组中String的下标。

练习11：(1) 将**AllTheColorsOfTheRainbow**这个示例改写成一个程序，然后编译、运行。

练习12：(2) 找出**HelloDate.java**的第二版本，也就是那个简单注释文档的示例。对该文件执行**javadoc**，然后通过Web浏览器观看运行结果。

练习13：(1) 通过**Javadoc**运行**Documentation1.java**，**Documentation2.java**和**Documentation3.java**，然后通过Web浏览器验证所产生的文档。

练习14：(1) 在前一个练习的文档中加入各项的HTML列表。 90

练习15：(1) 使用练习2的程序，加入注释文档。用javadoc提取此注释文档，并产生一个HTML文件，然后通过Web浏览器查看结果。

练习16：(1) 找到第5章中的**Overloading.java**示例，并为它加入javadoc文档。然后用**javadoc**提取此注释文档，并产生一个HTML文件，最后，通过Web浏览器查看结果。 91 ~ 92